

## Model Training – TF-IDF Vectorizer

- In this approach, we use the TF-IDF (Term Frequency–Inverse Document Frequency) method as our feature extraction technique.
- TF-IDF converts text into numerical feature vectors by assigning weights based on how important a word is within a document relative to the entire corpus.
- Before vectorization, the corpus is preprocessed by removing stopwords and applying necessary text-cleaning steps to reduce noise.
- After vectorization, each document is represented as a sparse TF-IDF vector, which is then used as input to various machine learning models.
- We will evaluate multiple ML models to compare their performance on TF-IDF features.

### ML Models Used:

- Logistic Regression
- Random Forest Classifier
- XGBoost Classifier
- MLP (implemented using both Keras and our own scratch implementation)

```
In [7]: import pandas as pd
import numpy as np
import random
import re
from datasets import load_dataset
```

```
In [8]: import tensorflow as tf
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    tf.config.experimental.set_memory_growth(gpus[0], True)
```

```
In [9]: #Loading dataset

dataset = load_dataset('ag_news')
train_df = dataset['train'].to_pandas()
test_df = dataset['test'].to_pandas()

X_train = train_df['text']
y_train = train_df['label']
X_test = test_df['text']
y_test = test_df['label']
```

```
In [10]: from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline
import time
```

```
In [11]: from sklearn.metrics import accuracy_score, f1_score, classification_report
```

## 1. Logistic Model

```
In [12]: %%time

start_cpu = time.process_time()
start=time.time()
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1, 2) # TF-IDF can benefit from bigrams
)

log_model = LogisticRegression(
    max_iter = 1000,
    random_state=42,
    multi_class='ovr')

pipeline = Pipeline(
    [
        ('vectorizer',vectorizer),
        ('classifier', log_model)
    ]
)

log_model = pipeline.fit(X_train,y_train)
end = time.time()
train_time = end-start
end_cpu = time.process_time()
cpu_time = end_cpu-start_cpu
print(train_time)
print(cpu_time)
```

C:\Users\moham\anaconda3\envs\cuda-env\lib\site-packages\sklearn\linear\_model\\_logistic.py:1256: FutureWarning: 'multi\_class' was deprecated in version 1.5 and will be removed in 1.7. Use OneVsRestClassifier(LogisticRegression(..)) instead. Leave it to its default value to avoid this warning.

```
warnings.warn(
10.159778356552124
10.0625
```

```
CPU times: total: 10.1 s
Wall time: 10.2 s
```

```
In [13]: %%time
y_pred_train = pipeline.predict(X_train)
```

CPU times: total: 4.34 s  
Wall time: 4.48 s

```
In [14]: %%time
start = time.time()
y_pred = pipeline.predict(X_test)
end = time.time()
inference_time = end-start
```

CPU times: total: 297 ms  
Wall time: 302 ms

```
In [15]: print("Training Accuracy")
train_accuracy = accuracy_score(y_train, y_pred_train)
train_f1 = f1_score(y_train, y_pred_train, average='weighted', zero_division=0)
train_report = classification_report(y_train,y_pred_train)
print('\n Training Accuracy',train_accuracy)
print('\n Training F1 Score',train_f1)
print('\n Training Classification Report',train_report)
```

Training Accuracy

Training Accuracy 0.9192833333333333

Training F1 Score 0.919100317557345

| Training Classification Report |      | precision | recall | f1-score | support |
|--------------------------------|------|-----------|--------|----------|---------|
| 0                              | 0.93 | 0.91      | 0.92   | 30000    |         |
| 1                              | 0.95 | 0.98      | 0.97   | 30000    |         |
| 2                              | 0.90 | 0.89      | 0.89   | 30000    |         |
| 3                              | 0.89 | 0.90      | 0.90   | 30000    |         |
| accuracy                       |      |           | 0.92   | 120000   |         |
| macro avg                      |      | 0.92      | 0.92   | 0.92     | 120000  |
| weighted avg                   |      | 0.92      | 0.92   | 0.92     | 120000  |

```
In [16]: print("Test Accuracy")
test_accuracy = accuracy_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
test_report = classification_report(y_test,y_pred)
print('\n Test Accuracy',test_accuracy)
print('\n Test F1 Score',test_f1)
print('\n Test Classification Report',test_report)
```

## Test Accuracy

Test Accuracy 0.9056578947368421

Test F1 Score 0.9053866266300983

| Test Classification Report |      |      |      | precision | recall | f1-score | support |
|----------------------------|------|------|------|-----------|--------|----------|---------|
| 0                          | 0.92 | 0.90 | 0.91 | 1900      |        |          |         |
| 1                          | 0.94 | 0.98 | 0.96 | 1900      |        |          |         |
| 2                          | 0.87 | 0.87 | 0.87 | 1900      |        |          |         |
| 3                          | 0.88 | 0.88 | 0.88 | 1900      |        |          |         |
| accuracy                   |      |      |      | 0.91      | 7600   |          |         |
| macro avg                  |      |      |      | 0.91      | 0.91   | 0.91     | 7600    |
| weighted avg               |      |      |      | 0.91      | 0.91   | 0.91     | 7600    |

## FLOPs:

The Formulas related to Logistic Regression

- $x = (W^T)X + b$
- $\hat{y} = \sigma(z) = \sigma(w^{\text{top}}x + b) = \frac{1}{1 + e^{-z}}$
- The FLOPs are  $2d - 1 \approx 2d$

Where d is the input dimension of the vector.

```
In [17]: vec = log_model.named_steps['vectorizer']
d = len(vec.get_feature_names_out())
print("Number of features (d):",d)
```

Number of features (d): 5000

```
In [18]: vec.get_feature_names_out()[:10]
```

```
Out[18]: array(['000', '000 jobs', '000 people', '04', '05', '10', '10 000',
               '10 million', '10 percent', '10 year'], dtype=object)
```

```
In [19]: results = {}
results[('Tf_IDF', 'Logistic Model')] = {
    'Train':
        {'accuracy': train_accuracy,
         'f1_score': train_f1,
         'report': train_report,
         'training_time': cpu_time},
    'Test':
        {'accuracy': test_accuracy,
         'f1_score': test_f1,
         'report': test_report,
         'inference_time': inference_time,
         "flops/sample": 2*d}}
```

In [20]: `print(results)`

```
{('Tf-IDF', 'Logistic Model'): {'Train': {'accuracy': 0.9192833333333333, 'f1_score': 0.919100317557345, 'report': '
precision    recall  f1-score   support\n\n      0      0.93      0.91      0.92    30000\n      1      0.98      0.97      0.98      30000\n      2      0.90      0.89      0.89      30000\n      3      0.89      0.90      0.90      30000\n\n accuracy\n0.92    120000\n macro avg      0.92      0.92      0.92    120000\nweighted avg\n0.92      0.92      0.92    120000\n'}, 'training_time': 10.0625}, 'Test': {'accuracy': 0.9056578947368421, 'f1_score': 0.9053866266300983, 'report': '
precision    recall  f1-score   support\n\n      0      0.92      0.90      0.91    19000\n      1      0.94      0.98      0.96    19000\n      2      0.87      0.87      0.87      19000\n      3      0.88      0.88      0.88      19000\n\n accuracy\n0.91      7600\n macro avg      0.91      0.91      0.91    7600\nweighted avg\n0.91      0.91      0.91    7600\n'}, 'inference_time': 0.3021509647369385, 'flops/sample': 10000}}}
```

In [ ]:

## 2. Random Forest Classifier.

In [21]: `%%time`

```
start=time.time()
start_cpu = time.process_time()
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1, 2) # TF-IDF can benefit from bigrams
)

rf_model = RandomForestClassifier(
    n_estimators = 100,
    max_depth = 10,
    random_state=42,
    n_jobs = -1)

pipeline = Pipeline(
    [
        ('vectorizer',vectorizer),
        ('classifier', rf_model)
    ]
)

rf_model = pipeline.fit(X_train,y_train)
end = time.time()
end_cpu = time.process_time()
train_time = end-start
cpu_time = end_cpu - start_cpu
print(train_time)
print(cpu_time)
```

9.806933164596558  
 17.015625  
 CPU times: total: 17 s  
 Wall time: 9.81 s

```
In [22]: %%time
y_pred_train = rf_model.predict(X_train)
```

CPU times: total: 5.44 s  
 Wall time: 4.6 s

```
In [23]: %%time
start = time.time()
y_pred = rf_model.predict(X_test)
end = time.time()
inference_time_rf = end-start
```

CPU times: total: 359 ms  
 Wall time: 341 ms

```
In [24]: print("Training Accuracy With Random Forest Model")
train_accuracy = accuracy_score(y_train, y_pred_train)
train_f1 = f1_score(y_train, y_pred_train, average='weighted', zero_division=0)
train_report = classification_report(y_train,y_pred_train)
print('\n Training Accuracy',train_accuracy)
print('\n Training F1 Score',train_f1)
print('\n Training Classification Report',train_report)
```

Training Accuracy With Random Forest Model

Training Accuracy 0.7788333333333334

Training F1 Score 0.7793000229365822

| Training Classification Report |      |      |      |       | precision | recall | f1-score | support |
|--------------------------------|------|------|------|-------|-----------|--------|----------|---------|
| 0                              | 0.81 | 0.78 | 0.80 | 30000 |           |        |          |         |
| 1                              | 0.85 | 0.87 | 0.86 | 30000 |           |        |          |         |
| 2                              | 0.85 | 0.65 | 0.74 | 30000 |           |        |          |         |
| 3                              | 0.65 | 0.81 | 0.72 | 30000 |           |        |          |         |
| accuracy                       |      |      |      |       | 0.78      | 120000 |          |         |
| macro avg                      |      |      |      |       | 0.79      | 0.78   | 0.78     | 120000  |
| weighted avg                   |      |      |      |       | 0.79      | 0.78   | 0.78     | 120000  |

```
In [25]: print("Test Accuracy with Random Forest Model")
test_accuracy = accuracy_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
test_report = classification_report(y_test,y_pred)
print('\n Test Accuracy',test_accuracy)
print('\n Test F1 Score',test_f1)
print('\n Test Classification Report',test_report)
```

## Test Accuracy with Random Forest Model

Test Accuracy 0.7686842105263157

Test F1 Score 0.7686727873630702

| Test Classification Report |   |      |      | precision | recall | f1-score | support |
|----------------------------|---|------|------|-----------|--------|----------|---------|
|                            | 0 | 0.80 | 0.79 | 0.79      |        |          | 1900    |
|                            | 1 | 0.85 | 0.86 | 0.85      |        |          | 1900    |
|                            | 2 | 0.84 | 0.62 | 0.71      |        |          | 1900    |
|                            | 3 | 0.64 | 0.81 | 0.71      |        |          | 1900    |
| accuracy                   |   |      |      | 0.77      |        |          | 7600    |
| macro avg                  |   |      |      | 0.78      | 0.77   | 0.77     | 7600    |
| weighted avg               |   |      |      | 0.78      | 0.77   | 0.77     | 7600    |

## FLOPs:

- Each tree in Random Forest has a depth, and this depth may vary based, The each step down the tree towards leave can be counted as a FLOP, so at maximum the maximum FLOPs from each tree is its depth. so to get FLOPs for the whole Random Forest model we can directly use the average of depths of all tree.

```
In [26]: rf = rf_model.named_steps['classifier']
tree_depths = np.array([estimator.tree_.max_depth for estimator in rf.estimators_])
avg_max_depth = tree_depths.mean()
avg_flops_rf = rf.n_estimators * avg_max_depth
print("Average flops:", avg_flops_rf)
```

Average flops: 1000.0

In [ ]:

```
In [27]: results[('Tf_IDF', 'Random Forest Model')] = {
            'Train':
                {'accuracy': train_accuracy,
                 'f1_score': train_f1,
                 'report': train_report,
                 'training_time': cpu_time
                },
            'Test':
                {'accuracy': test_accuracy,
                 'f1_score': test_f1,
                 'report': test_report,
                 'inference_time': inference_time_rf,
                 'flops/sample': avg_flops_rf
                }
        }
```

In [ ]:

### 3. XGBoost Classifier.

```
In [28]: %%time

start_cpu = time.process_time()
start=time.time()
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1, 2) # TF-IDF can benefit from bigrams
)

xgb_model = XGBClassifier(
    n_estimators = 100,
    max_depth = 10,
    random_state=42,
    n_jobs = -1)

pipeline = Pipeline(
    [
        ('vectorizer',vectorizer),
        ('classifier', xgb_model)
    ]
)

xgb_model = pipeline.fit(X_train,y_train)
end = time.time()
cpu_end = time.process_time()
train_time = end-start
cpu_time = cpu_end - start_cpu
print(train_time)
print(cpu_time)
```

189.3609745502472

2439.328125

CPU times: total: 40min 39s

Wall time: 3min 9s

The Cell time is 27.3 seconds, and the CPU Time is 5min 20s, the difference is because the model is trained parallelly on 10 CPU Cores ,because my pc has 10 cores ,this will vary for each pc based on cores and other task running on the pc.

```
In [29]: %%time

y_pred_train = xgb_model.predict(X_train)
```

CPU times: total: 9.39 s

Wall time: 4.6 s

```
In [30]: %%time

start = time.time()
y_pred = xgb_model.predict(X_test)
end = time.time()
inference_time_xgb = end-start
```



CPU times: total: 1.77 s

Wall time: 376 ms

```
In [31]: print("Training Accuracy With XG Boost Model")
train_accuracy = accuracy_score(y_train, y_pred_train)
train_f1 = f1_score(y_train, y_pred_train, average='weighted', zero_division=0)
train_report = classification_report(y_train,y_pred_train)
print('\n Training Accuracy',train_accuracy)
print('\n Training F1 Score',train_f1)
print('\n Training Classification Report',train_report)
```

Training Accuracy With XG Boost Model

Training Accuracy 0.9362916666666666

Training F1 Score 0.936206407541604

| Training Classification Report | precision | recall | f1-score | support |
|--------------------------------|-----------|--------|----------|---------|
| 0                              | 0.95      | 0.93   | 0.94     | 30000   |
| 1                              | 0.95      | 0.98   | 0.97     | 30000   |
| 2                              | 0.93      | 0.92   | 0.92     | 30000   |
| 3                              | 0.91      | 0.92   | 0.92     | 30000   |
| accuracy                       |           |        | 0.94     | 120000  |
| macro avg                      | 0.94      | 0.94   | 0.94     | 120000  |
| weighted avg                   | 0.94      | 0.94   | 0.94     | 120000  |

```
In [32]: print("Test Accuracy with XG Boost Model")
test_accuracy = accuracy_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
test_report = classification_report(y_test,y_pred)
print('\n Test Accuracy',test_accuracy)
print('\n Test F1 Score',test_f1)
print('\n Test Classification Report',test_report)
```

Test Accuracy with XG Boost Model

Test Accuracy 0.9030263157894737

Test F1 Score 0.9027899998143843

| Test Classification Report | precision | recall | f1-score | support |
|----------------------------|-----------|--------|----------|---------|
| 0                          | 0.92      | 0.90   | 0.91     | 1900    |
| 1                          | 0.93      | 0.97   | 0.95     | 1900    |
| 2                          | 0.88      | 0.86   | 0.87     | 1900    |
| 3                          | 0.87      | 0.88   | 0.88     | 1900    |
| accuracy                   |           |        | 0.90     | 7600    |
| macro avg                  | 0.90      | 0.90   | 0.90     | 7600    |
| weighted avg               | 0.90      | 0.90   | 0.90     | 7600    |

```
In [33]: xgb_classifier = xgb_model.named_steps['classifier']
         booster = xgb_classifier.get_booster()
         total_trees = len(booster.get_dump())
         flops_per_sample_xgb = total_trees * xgb_classifier.get_params()['max_depth']
         print(f"Approx FLOPs per sample: {flops_per_sample_xgb}")
```

Approx FLOPs per sample: 4000

```
In [34]: results[('Tf_IDF', 'XG Boost Model')] = {
        'Train':
            {'accuracy': train_accuracy,
             'f1_score': train_f1,
             'report': train_report,
             'training_time': cpu_time
            },
        'Test':
            {'accuracy': test_accuracy,
             'f1_score': test_f1,
             'report': test_report,
             'inference_time': inference_time_xgb,
             'flops/sample': flops_per_sample_xgb}}
```

In [ ]:

## 4. MLP Model.

```
In [35]: from tensorflow.keras.models import Sequential
         from tensorflow.keras.layers import Dense, Dropout
         from tensorflow.keras.utils import to_categorical
         from sklearn.neural_network import MLPClassifier
         from tensorflow.keras import regularizers
         import tensorflow as tf
```

```
In [39]: %%time

         vectorizer = TfidfVectorizer(
             max_features=5000,
             stop_words='english',
             ngram_range=(1, 2) # TF-IDF can benefit from bigrams
         )

         X_train_vec = vectorizer.fit_transform(X_train)
         X_test_vec = vectorizer.transform(X_test)

         X_train_vec = X_train_vec.toarray()
         X_test_vec = X_test_vec.toarray()

         num_classes = len(np.unique(y_train))
         y_train_cat = to_categorical(y_train, num_classes)
         y_test_cat = to_categorical(y_test, num_classes)

         input_dim = X_train_vec.shape[1]

         mlp_model = Sequential([
```

```

Dense(256, activation='relu', input_dim=input_dim, kernel_regularizer=regularize
Dropout(0.2),
Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
Dropout(0.3),
Dense(4, activation='softmax')
])

print("Model Summary:", mlp_model.summary())
mlp_model.compile(
    loss= 'categorical_crossentropy',
    optimizer = 'adam',
    metrics=['accuracy']
)

```

Model: "sequential"

| Layer (type)        | Output Shape | Param # |
|---------------------|--------------|---------|
| dense (Dense)       | (None, 256)  | 1280256 |
| dropout (Dropout)   | (None, 256)  | 0       |
| dense_1 (Dense)     | (None, 128)  | 32896   |
| dropout_1 (Dropout) | (None, 128)  | 0       |
| dense_2 (Dense)     | (None, 4)    | 516     |

=====

Total params: 1,313,668  
Trainable params: 1,313,668  
Non-trainable params: 0

Model Summary: None  
CPU times: total: 10.4 s  
Wall time: 10.7 s

```

In [40]: %%time

start_cpu = time.process_time()
start=time.time()
history = mlp_model.fit(
    X_train_vec, y_train_cat,
    epochs=10,
    batch_size=128,
    verbose=1
)

end = time.time()
cpu_end = time.process_time()
train_time = end-start
cpu_time = cpu_end - start_cpu
print(train_time)
print(cpu_time)

```

```

Epoch 1/10
938/938 [=====] - 6s 4ms/step - loss: 0.5864 - accuracy: 0.8794
Epoch 2/10
938/938 [=====] - 3s 3ms/step - loss: 0.4890 - accuracy: 0.8937
Epoch 3/10
938/938 [=====] - 3s 3ms/step - loss: 0.4631 - accuracy: 0.8968
Epoch 4/10
938/938 [=====] - 2s 3ms/step - loss: 0.4406 - accuracy: 0.9003
Epoch 5/10
938/938 [=====] - 2s 2ms/step - loss: 0.4235 - accuracy: 0.9031
Epoch 6/10
938/938 [=====] - 2s 2ms/step - loss: 0.4091 - accuracy: 0.9051
Epoch 7/10
938/938 [=====] - 2s 3ms/step - loss: 0.3994 - accuracy: 0.9070
Epoch 8/10
938/938 [=====] - 2s 2ms/step - loss: 0.3931 - accuracy: 0.9081
Epoch 9/10
938/938 [=====] - 2s 2ms/step - loss: 0.3815 - accuracy: 0.9106
Epoch 10/10
938/938 [=====] - 2s 2ms/step - loss: 0.3797 - accuracy: 0.9114
33.400989055633545
49.234375
CPU times: total: 49.2 s
Wall time: 33.4 s

```

```

In [42]: with tf.device('/CPU:0'):
          y_pred_train_probs = mlp_model.predict(X_train_vec)
          y_pred_train = np.argmax(y_pred_train_probs, axis=1)
          y_true_train = np.argmax(y_train_cat, axis=1)

3750/3750 [=====] - 16s 4ms/step

```

```

In [43]: start = time.time()
          with tf.device('/CPU:0'):
              y_pred_probs = mlp_model.predict(X_test_vec)
          end = time.time()

          inference_time_mlp = (end - start)
          print(f"Average Inference Time sample: {inference_time*1000:.4f} ms")

          y_pred = np.argmax(y_pred_probs, axis=1)
          y_true_test = np.argmax(y_test_cat, axis=1)

238/238 [=====] - 1s 4ms/step
Average Inference Time sample: 302.1510 ms

```

```
In [44]: print("Training Accuracy With MLP Model")
train_accuracy = accuracy_score(y_true_train, y_pred_train)
train_f1 = f1_score(y_true_train, y_pred_train, average='weighted', zero_division=0)
train_report = classification_report(y_true_train,y_pred_train)
print('\n Training Accuracy',train_accuracy)
print('\n Training F1 Score',train_f1)
print('\n Training Classification Report',train_report)
```

Training Accuracy With MLP Model

Training Accuracy 0.9304166666666667

Training F1 Score 0.930338424419159

| Training Classification Report | precision | recall | f1-score | support |
|--------------------------------|-----------|--------|----------|---------|
| 0                              | 0.95      | 0.91   | 0.93     | 30000   |
| 1                              | 0.96      | 0.99   | 0.97     | 30000   |
| 2                              | 0.89      | 0.92   | 0.91     | 30000   |
| 3                              | 0.92      | 0.90   | 0.91     | 30000   |
| accuracy                       |           |        | 0.93     | 120000  |
| macro avg                      | 0.93      | 0.93   | 0.93     | 120000  |
| weighted avg                   | 0.93      | 0.93   | 0.93     | 120000  |

```
In [45]: print("Test Accuracy with MLP Model")
test_accuracy = accuracy_score(y_true_test, y_pred)
test_f1 = f1_score(y_true_test, y_pred, average='weighted', zero_division=0)
test_report = classification_report(y_true_test,y_pred)
print('\n Test Accuracy',test_accuracy)
print('\n Test F1 Score',test_f1)
print('\n Test Classification Report',test_report)
```

Test Accuracy with MLP Model

Test Accuracy 0.9052631578947369

Test F1 Score 0.9051564663576387

| Test Classification Report | precision | recall | f1-score | support |
|----------------------------|-----------|--------|----------|---------|
| 0                          | 0.94      | 0.89   | 0.91     | 1900    |
| 1                          | 0.94      | 0.98   | 0.96     | 1900    |
| 2                          | 0.85      | 0.89   | 0.87     | 1900    |
| 3                          | 0.90      | 0.86   | 0.88     | 1900    |
| accuracy                   |           |        | 0.91     | 7600    |
| macro avg                  | 0.91      | 0.91   | 0.91     | 7600    |
| weighted avg               | 0.91      | 0.91   | 0.91     | 7600    |

```
In [46]: def analytical_flops(model):
flops = 0
for layer in model.layers:
```

```

        if isinstance(layer, tf.keras.layers.Dense):
            in_features = layer.input_shape[-1]
            out_features = layer.output_shape[-1]
            flops += 2 * in_features * out_features
    return flops

```

```

flops = analytical_flops(mlp_model)
print(f"Analytical FLOPs: {flops} FLOPs")

```

Analytical FLOPs: 2626560 FLOPs

```

In [47]: results[('Tf_IDF', 'MLP Model')] = {
    'Train':
        {'accuracy': train_accuracy,
         'f1_score': train_f1,
         'report': train_report,
         'training_time': cpu_time
        },
    'Test':
        {'accuracy': test_accuracy,
         'f1_score': test_f1,
         'report': test_report,
         'inference_time': inference_time_mlp,
         'flops/sample': flops}}

```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

## MLP Model - Scratch Code.

```

In [48]: import numpy as np

```

```

In [49]: num_features = X_train_vec.shape[1]
    num_classes = len(np.unique(y_train))

    def one_hot_encode(y, num_classes):
        return np.eye(num_classes)[y]

    y_train_oh = one_hot_encode(y_train, num_classes)
    y_test_oh = one_hot_encode(y_test, num_classes)

```

```

In [50]: y_train_oh

```

```
Out[50]: array([[0., 0., 1., 0.],
               [0., 0., 1., 0.],
               [0., 0., 1., 0.],
               ...,
               [0., 1., 0., 0.],
               [0., 1., 0., 0.],
               [0., 1., 0., 0.]])
```

```
In [51]: y_train
```

```
Out[51]: 0      2
         1      2
         2      2
         3      2
         4      2
         ..
119995    0
119996    1
119997    1
119998    1
119999    1
Name: label, Length: 120000, dtype: int64
```

```
In [52]: num_features
```

```
Out[52]: 5000
```

```
In [79]: X_train_vec
```

```
Out[79]: array([[0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.]])
```

```
In [53]: def init_params(input_dim, hidden1, hidden2, output_dim):
         params = {
             'W1' : np.random.randn(input_dim,hidden1) * np.sqrt(2./input_dim),
             'b1' : np.zeros((1,hidden1)),
             'W2' : np.random.randn(hidden1, hidden2) * np.sqrt(2./hidden1),
             'b2' : np.zeros((1,hidden2)),
             'W3' : np.random.randn(hidden2, output_dim) * np.sqrt(2./hidden2),
             'b3' : np.zeros((1,output_dim))
         }
         return params
```

```
params = init_params(num_features, 256, 128, num_classes)
```

```
In [54]: def relu(Z):
         return np.maximum(0,Z)
         def relu_deriv(Z):
             return (Z>0).astype(float)
         def softmax(Z):
```

```
expZ = np.exp(Z-np.max(Z,axis=1, keepdims=True)) # we are essentially normalizing
return expZ / np.sum(expZ, axis=1, keepdims=True)
```

```
In [55]: def forward(X, params):
    Z1 = X @ params['W1'] + params['b1']
    A1 = relu(Z1)
    Z2 = A1 @ params['W2'] + params['b2']
    A2 = relu(Z2)
    Z3 = A2 @ params['W3'] + params['b3']
    A3 = softmax(Z3)

    cache = (X, Z1, A1, Z2, A2, Z3, A3)
    return A3, cache

def backward(params, cache, y_true):
    X, Z1, A1, Z2, A2, Z3, A3 = cache
    m = X.shape[0]

    dZ3 = A3 - y_true
    dW3 = (A2.T @ dZ3) / m
    db3 = np.sum(dZ3, axis=0, keepdims=True) / m

    dA2 = dZ3 @ params['W3'].T
    dZ2 = dA2 * relu_deriv(Z2)
    dW2 = (A1.T @ dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m

    dA1 = dZ2 @ params['W2'].T
    dZ1 = dA1 * relu_deriv(Z1)
    dW1 = (X.T @ dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m

    grads = {
        'dW1': dW1, 'db1': db1,
        'dW2': dW2, 'db2': db2,
        'dW3': dW3, 'db3': db3
    }

    return grads
```

```
In [56]: def compute_loss(y_true, y_pred):
    m = y_true.shape[0]
    loss = -np.sum(y_true * np.log(y_pred + 1e-9)) / m
    return loss
```

```
In [57]: def update_params(params, grads, lr):
    for key in params.keys():
        params[key] -= lr*grads['d' + key]

    return params
```

```
In [58]: def train(X, y, params, epochs=10, lr=0.01, batch_size=128):
    m = X.shape[0]
    for epoch in range(epochs):
        perm = np.random.permutation(m)
```



```

X_shuffled, y_shuffled = X[perm], y[perm]

losses = []
for i in range(0, m, batch_size):
    X_batch = X_shuffled[i:i+batch_size]
    y_batch = y_shuffled[i:i+batch_size]

    y_pred, cache = forward(X_batch, params)
    loss = compute_loss(y_batch, y_pred)
    grads = backward(params, cache, y_batch)
    params = update_params(params, grads, lr)
    losses.append(loss)

print(f"Epoch {epoch+1}/{epochs}, Loss = {np.mean(losses):.4f}")
return params

```

```

In [59]: %%time
start = time.time()
cpu_start = time.process_time()

def predict(X, params):
    y_pred, _ = forward(X, params)
    return np.argmax(y_pred, axis=1)

# train
trained_params = train(X_train_vec, y_train_oh, params, epochs=10, lr=0.01)

end = time.time()
cpu_end = time.process_time()
cpu_time = cpu_end - cpu_start
cell_time = end - start
print(cpu_time)

```

```

Epoch 1/10, Loss = 1.3782
Epoch 2/10, Loss = 1.3480
Epoch 3/10, Loss = 1.2742
Epoch 4/10, Loss = 1.1128
Epoch 5/10, Loss = 0.8579
Epoch 6/10, Loss = 0.6273
Epoch 7/10, Loss = 0.4877
Epoch 8/10, Loss = 0.4131
Epoch 9/10, Loss = 0.3713
Epoch 10/10, Loss = 0.3454
1473.25
CPU times: total: 24min 33s
Wall time: 3min 46s

```

```

In [60]: %%time
start_inf_time = time.process_time()
y_pred = predict(X_test_vec, trained_params)
end_inf_time = time.process_time()
inf_time_smlp = end_inf_time - start_inf_time

```

```

CPU times: total: 3.31 s
Wall time: 243 ms

```

```
In [61]: y_pred_train = predict(X_train_vec, trained_params)
```

```
In [62]: np.array(y_pred)
```

```
Out[62]: array([2, 3, 3, ..., 1, 2, 3], dtype=int64)
```

```
In [63]: y_test
```

```
Out[63]: 0      2
         1      3
         2      3
         3      3
         4      3
         ..
       7595    0
       7596    1
       7597    1
       7598    2
       7599    2
       Name: label, Length: 7600, dtype: int64
```

```
In [64]: print("Training Accuracy With Scratch MLP Model")
         train_accuracy = accuracy_score(y_train, y_pred_train)
         train_f1 = f1_score(y_train, y_pred_train, average='weighted', zero_division=0)
         train_report = classification_report(y_train,y_pred_train)
         print('\n Training Accuracy',train_accuracy)
         print('\n Training F1 Score',train_f1)
         print('\n Training Classification Report',train_report)
```

Training Accuracy With Scratch MLP Model

Training Accuracy 0.8963833333333333

Training F1 Score 0.8960343561375421

| Training Classification Report |      | precision | recall | f1-score | support |
|--------------------------------|------|-----------|--------|----------|---------|
| 0                              | 0.89 | 0.90      | 0.90   | 0.90     | 30000   |
| 1                              | 0.94 | 0.97      | 0.96   | 0.96     | 30000   |
| 2                              | 0.88 | 0.85      | 0.86   | 0.86     | 30000   |
| 3                              | 0.87 | 0.86      | 0.87   | 0.87     | 30000   |
| accuracy                       |      |           | 0.90   |          | 120000  |
| macro avg                      | 0.90 | 0.90      | 0.90   |          | 120000  |
| weighted avg                   | 0.90 | 0.90      | 0.90   |          | 120000  |

```
In [65]: print("Test Accuracy with Scratch MLP Model")
         test_accuracy = accuracy_score(y_test, y_pred)
         test_f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
         test_report = classification_report(y_test,y_pred)
         print('\n Test Accuracy',test_accuracy)
         print('\n Test F1 Score',test_f1)
         print('\n Test Classification Report',test_report)
```

## Test Accuracy with Scratch MLP Model

Test Accuracy 0.8880263157894737

Test F1 Score 0.8875493127726304

| Test Classification Report |      |      |      | precision | recall | f1-score | support |
|----------------------------|------|------|------|-----------|--------|----------|---------|
| 0                          | 0.89 | 0.90 | 0.89 | 0.89      | 1900   |          |         |
| 1                          | 0.94 | 0.97 | 0.95 | 0.95      | 1900   |          |         |
| 2                          | 0.86 | 0.84 | 0.85 | 0.85      | 1900   |          |         |
| 3                          | 0.86 | 0.85 | 0.85 | 0.85      | 1900   |          |         |
| accuracy                   |      |      |      | 0.89      | 7600   |          |         |
| macro avg                  |      |      |      | 0.89      | 0.89   | 0.89     | 7600    |
| weighted avg               |      |      |      | 0.89      | 0.89   | 0.89     | 7600    |

For a fully connected (dense) layer, each output neuron performs:

FLOPs per layer =  $2 * (\text{input size}) * (\text{output size})$

- Layer 1 FLOPs 1 =  $2 \times N \times d \times h$
- Layer 2 FLOPs 2 =  $2 \times N \times h$
- Output layer FLOPs 3 =  $2 \times N \times h$

Note we will be calculating only the forward pass FLOPs

```
In [66]: def mlp_flops(d, h1, h2, c, N=1):
          forward = 2 * N * (d*h1 + h1*h2 + h2*c)
          return forward

          flops = mlp_flops(num_features, 256,128,4,128)
          print("FLOPs per sample of forward pass", flops / 128.0)
```

FLOPs per sample of forward pass 2626560.0

```
In [67]: results[('Tf_IDF', 'Scratch MLP Model')] = {
          'Train':
            {'accuracy': train_accuracy,
             'f1_score': train_f1,
             'report': train_report,
             'training_time': cpu_time
            },
          'Test':
            {'accuracy': test_accuracy,
             'f1_score': test_f1,
             'report': test_report,
             'inference_time': inf_time_smlp,
             'flops/sample': flops/128.0}}
```

```
In [68]: import pandas as pd

          # flatten nested dict into a list of rows
```

```

rows = []

for (vectorizer, model), metrics in results.items():
    row = {
        "Vectorizer": vectorizer,
        "Model": model,
        "Train_Accuracy": metrics["Train"]["accuracy"],
        "Train_F1": metrics["Train"]["f1_score"],
        "Test_Accuracy": metrics["Test"]["accuracy"],
        "Test_F1": metrics["Test"]["f1_score"],
        "Inference_Time": metrics["Test"].get("inference_time", None),
        "FLOPs_per_Sample": metrics["Test"].get("flops/sample", None),
        "Training_Time": metrics['Train'].get('training_time', None)
    }
    rows.append(row)

# create DataFrame
df_results = pd.DataFrame(rows)

# optional: set a clean display order
df_results = df_results[
    ["Vectorizer", "Model", "Train_Accuracy", "Train_F1",
     "Test_Accuracy", "Test_F1", "Inference_Time", "FLOPs_per_Sample", "Training_Tim
]

print(df_results)

```

|   | Vectorizer | Model               | Train_Accuracy | Train_F1 | Test_Accuracy \ |
|---|------------|---------------------|----------------|----------|-----------------|
| 0 | Tf_IDF     | Logistic Model      | 0.919283       | 0.919100 | 0.905658        |
| 1 | Tf_IDF     | Random Forest Model | 0.778833       | 0.779300 | 0.768684        |
| 2 | Tf_IDF     | XG Boost Model      | 0.936292       | 0.936206 | 0.903026        |
| 3 | Tf_IDF     | MLP Model           | 0.930417       | 0.930338 | 0.905263        |
| 4 | Tf_IDF     | Scratch MLP Model   | 0.896383       | 0.896034 | 0.888026        |

  

|   | Test_F1  | Inference_Time | FLOPs_per_Sample | Training_Time |
|---|----------|----------------|------------------|---------------|
| 0 | 0.905387 | 0.302151       | 10000.0          | 10.062500     |
| 1 | 0.768673 | 0.341379       | 1000.0           | 17.015625     |
| 2 | 0.902790 | 0.376454       | 4000.0           | 2439.328125   |
| 3 | 0.905156 | 1.405875       | 2626560.0        | 49.234375     |
| 4 | 0.887549 | 3.312500       | 2626560.0        | 1473.250000   |

In [69]: df\_results

Out[69]:

|   | Vectorizer | Model               | Train_Accuracy | Train_F1 | Test_Accuracy | Test_F1  | Inference_Time |
|---|------------|---------------------|----------------|----------|---------------|----------|----------------|
| 0 | Tf_IDF     | Logistic Model      | 0.919283       | 0.919100 | 0.905658      | 0.905387 | 0.302151       |
| 1 | Tf_IDF     | Random Forest Model | 0.778833       | 0.779300 | 0.768684      | 0.768673 | 0.341379       |
| 2 | Tf_IDF     | XG Boost Model      | 0.936292       | 0.936206 | 0.903026      | 0.902790 | 0.376454       |
| 3 | Tf_IDF     | MLP Model           | 0.930417       | 0.930338 | 0.905263      | 0.905156 | 1.405875       |
| 4 | Tf_IDF     | Scratch MLP Model   | 0.896383       | 0.896034 | 0.888026      | 0.887549 | 3.312500       |


In [70]: `df_results.to_csv('datasets/results_tfidf.csv')`In [71]: `df_bow = pd.read_csv('datasets/results.csv')`  
`df = pd.concat([df_bow, df_results], axis=0, ignore_index=True)`In [75]: `df = df.drop('Unnamed: 0',axis=1)`In [77]: `df`

Out[77]:

|   | Vectorizer | Model               | Train_Accuracy | Train_F1 | Test_Accuracy | Test_F1  | Inference_Time |
|---|------------|---------------------|----------------|----------|---------------|----------|----------------|
| 0 | BOW        | Logistic Model      | 0.929450       | 0.929328 | 0.902237      | 0.902075 | 0.136527       |
| 1 | BOW        | Random Forest Model | 0.789375       | 0.788256 | 0.781579      | 0.779980 | 0.210410       |
| 2 | BOW        | XG Boost Model      | 0.930625       | 0.930504 | 0.904605      | 0.904392 | 0.184776       |
| 3 | BOW        | MLP Model           | 0.949333       | 0.949257 | 0.913026      | 0.912820 | 1.236788       |
| 4 | BOW        | Scratch MLP Model   | 0.920692       | 0.920560 | 0.902895      | 0.902761 | 3.453125       |
| 5 | Tf_IDF     | Logistic Model      | 0.919283       | 0.919100 | 0.905658      | 0.905387 | 0.302151       |
| 6 | Tf_IDF     | Random Forest Model | 0.778833       | 0.779300 | 0.768684      | 0.768673 | 0.341379       |
| 7 | Tf_IDF     | XG Boost Model      | 0.936292       | 0.936206 | 0.903026      | 0.902790 | 0.376454       |
| 8 | Tf_IDF     | MLP Model           | 0.930417       | 0.930338 | 0.905263      | 0.905156 | 1.405875       |
| 9 | Tf_IDF     | Scratch MLP Model   | 0.896383       | 0.896034 | 0.888026      | 0.887549 | 3.312500       |

In [76]: `df.to_csv('datasets/results.csv')`

In [ ]: